

GSOC EXERCISES

Exercise 1 and 2:-

https://cernbox.cern.ch/pdf-viewer/public/jrsV36vUll9Lx8R/GSoC-ML4EP-SOFIE-Introduction_Exercises.pdf?contextRouteName=files-public-link&contextRouteParams.driveAliasAndItem=public%2FjrsV36vUll9Lx8R&contextRouteQuery.fileId=jrsV36vUll9Lx8R

Exercise 3,4 and 5:-

https://cernbox.cern.ch/pdf-viewer/public/pzUA3ZQc29WELaN/2026-GSoC-SOFIE_Exercise_Heterogeneous_Inference.pdf?contextRouteName=files-public-link&contextRouteParams.driveAliasAndItem=public%2FpzUA3ZQc29WELaN&contextRouteQuery.fileId=pzUA3ZQc29WELaN

Exercise 1 :-

Setup:-

```
1.sudo apt update
sudo apt install -y \
git \
cmake \
build-essential \
gcc \
g++ \
gfortran \
libx11-dev \
libxpm-dev \
libxft-dev \
libxext-dev \
python3 \
python3-pip \
```

```
python3-dev \  
libssl-dev \  
libxml2-dev \  
libbz2-dev \  
liblzma-dev \  
libzstd-dev \  
libgl1-mesa-dev \  
libglu1-mesa-dev \  
protobuf-compiler \  
libprotobuf-dev  
sudo apt install -y \  
protobuf-compiler \  
libprotobuf-dev \  
libonnx-dev \  
ninja-build \  
Python3-dev
```

```
2. python3 -m venv rootenv  
source rootenv/bin/activate  
pip install numpy pandas  
Pip install scikit-learn  
pip install tensorflow  
pip install torch torchvision  
pip install keras  
pip install onnx onnxruntime
```

```
3. git clone https://github.com/ajatshatru01/root.git  
cd root  
git remote add upstream https://github.com/root-project/root.git  
git checkout -b gsoc-2026-utsav  
mkdir build  
cd build
```

```
4. sudo apt install ninja-build
```

```
cmake .. -G Ninja \  
-DCMAKE_BUILD_TYPE=Release \  
-Dtmva=ON \  
-Dtmva-sofie=ON \  
-Dtmva-pymva=ON \  

```

```
-Dpyroot=ON \
-Droofit=ON \
-Dmathmore=ON \
-Ddataframe=ON \
-Dimt=ON \
-Dopengl=ON \
-Dx11=ON \
-Dbuiltin_xrootd=ON \
-DPYTHON_EXECUTABLE=$(which python)
```

ninja -j2

5. source bin/[thisroot.sh](#)

CUDA_VISIBLE_DEVICES="" root

Exercise 2 :-

- Running tutorial TMVA_Higgs_Classification.C

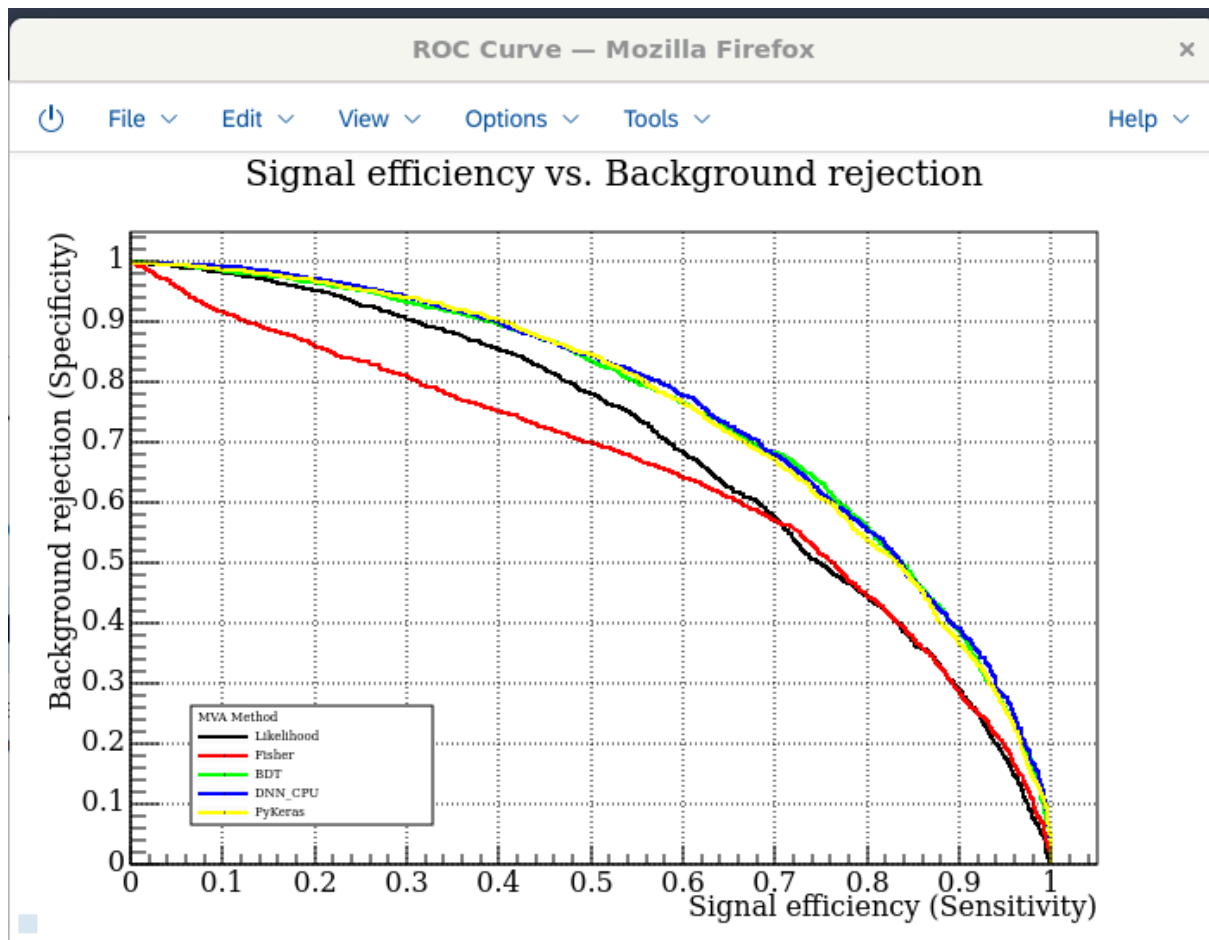
root [0] .x

../tutorials/machine_learning/TMVA_Higgs_Classification.C

```
Epoch 15: finished saving model to Higgs_trained_model.keras
112/112 ————— 1s 5ms/step - accuracy: 0.6842 - loss: 0.5860 - val_accuracy: 0.6768 - val_loss: 0.5921
Epoch 16/20
 95/112 ————— 0s 2ms/step - accuracy: 0.6855 - loss: 0.5835
Epoch 16: val_loss did not improve from 0.59207
112/112 ————— 0s 3ms/step - accuracy: 0.6862 - loss: 0.5841 - val_accuracy: 0.6629 - val_loss: 0.5998
Epoch 17/20
103/112 ————— 0s 4ms/step - accuracy: 0.6893 - loss: 0.5756
Epoch 17: val_loss did not improve from 0.59207
112/112 ————— 1s 4ms/step - accuracy: 0.6845 - loss: 0.5832 - val_accuracy: 0.6811 - val_loss: 0.5927
Epoch 18/20
 99/112 ————— 0s 2ms/step - accuracy: 0.6755 - loss: 0.5879
Epoch 18: val_loss did not improve from 0.59207
112/112 ————— 0s 4ms/step - accuracy: 0.6832 - loss: 0.5805 - val_accuracy: 0.6732 - val_loss: 0.5933
Epoch 19/20
 99/112 ————— 0s 4ms/step - accuracy: 0.6927 - loss: 0.5724
Epoch 19: val_loss improved from 0.59207 to 0.59045, saving model to Higgs_trained_model.keras

Epoch 19: finished saving model to Higgs_trained_model.keras
112/112 ————— 1s 5ms/step - accuracy: 0.6873 - loss: 0.5780 - val_accuracy: 0.6821 - val_loss: 0.5904
Epoch 20/20
101/112 ————— 0s 2ms/step - accuracy: 0.6958 - loss: 0.5714
Epoch 20: val_loss improved from 0.59045 to 0.58798, saving model to Higgs_trained_model.keras

Epoch 20: finished saving model to Higgs_trained_model.keras
112/112 ————— 0s 3ms/step - accuracy: 0.6876 - loss: 0.5794 - val_accuracy: 0.6771 - val_loss: 0.5880
```



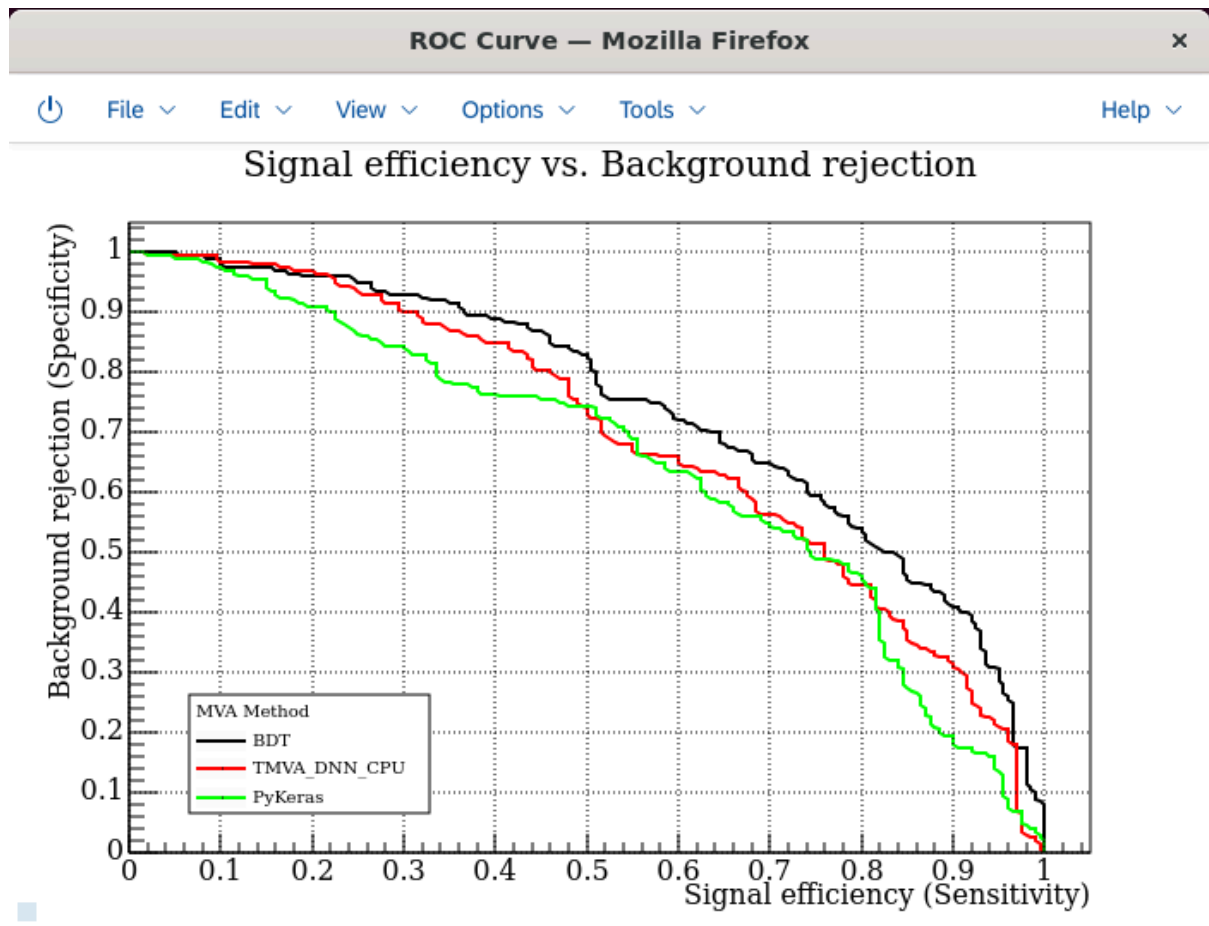
- Running tutorial TMVA_CNN_Classification.C

root [1] .x

../tutorials/machine_learning/TMVA_CNN_Classification.C

Model: "sequential"

Layer (type)	Output Shape	Param #
reshape (Reshape)	(None, 16, 16, 1)	0
conv2d (Conv2D)	(None, 16, 16, 10)	100
batch_normalization (BatchNormalization)	(None, 16, 16, 10)	40
conv2d_1 (Conv2D)	(None, 16, 16, 10)	910
max_pooling2d (MaxPooling2D)	(None, 15, 15, 10)	0
flatten (Flatten)	(None, 2250)	0
dense (Dense)	(None, 256)	576,256
dense_1 (Dense)	(None, 2)	514



- **Running SOFIE tutorials :-**

1.

(rootenv)

utsav@AjatShatru:~/root/tutorials/machine_learning

\$ root TMVA_SOFIE_ONNX.C

Sample output:-

```

//----- Gemm op_8 { 16 , 50 } * { 50 , 50 } -> { 16 , 50 }
for (size_t j = 0; j < 16; j++) {
    size_t y_index = 50 * j;
    for (size_t k = 0; k < 50; k++) {
        tensor_38[y_index + k] = tensor_16bias[k];
    }
}
TMVA::Experimental::SOFIE::Gemm_Call(tensor_38, true, false, 50, 16, 50, 1, tensor_16weight, tensor_36, 1, nullptr);
for (int id = 0; id < 800 ; id++){
    tensor_38[id] = ((tensor_38[id] > 0 )? tensor_38[id] : 0);
}

//----- Gemm op_9 { 16 , 50 } * { 10 , 50 } -> { 16 , 10 }
for (size_t j = 0; j < 16; j++) {
    size_t y_index = 10 * j;
    for (size_t k = 0; k < 10; k++) {
        tensor_39[y_index + k] = tensor_18bias[k];
    }
}
TMVA::Experimental::SOFIE::Gemm_Call(tensor_39, true, false, 10, 16, 50, 1, tensor_18weight, tensor_38, 1, nullptr);
using TMVA::Experimental::SOFIE::UTILITY::FillOutput;

FillOutput(tensor_39, output_tensor_39, 160);
}

std::vector<float> infer(float const* tensor_input1){
    std::vector<float > output_tensor_39;
    doInfer(tensor_input1, output_tensor_39 );
    return {output_tensor_39};
}
}; // end of Session
} //TMVA_SOFIE_Linear_16
#endif // ROOT_TMVA_SOFIE_LINEAR_16

```

2. (rootenv)

utsav@AjatShatru:~/root/tutorials/machine_learning\$ root TMVA_SOFIE_PyTorch.C

Sample output:-

```

//----- RELU
for (int id = 0; id < 32 ; id++){
    tensor_result[id] = ((tensor_input0[id] > 0 )? tensor_input0[id] : 0);
}

//----- Gemm op_2 { 2 , 16 } * { 8 , 16 } -> { 2 , 8 }
for (size_t j = 0; j < 2; j++) {
    size_t y_index = 8 * j;
    for (size_t k = 0; k < 8; k++) {
        tensor_input2[y_index + k] = tensor_2bias[k];
    }
}
TMVA::Experimental::SOFIE::Gemm_Call(tensor_input2, true, false, 8, 2, 16, 1, tensor_2weight, tensor_result, 1, nullptr);

//----- RELU
for (int id = 0; id < 16 ; id++){
    tensor_result3[id] = ((tensor_input2[id] > 0 )? tensor_input2[id] : 0);
}
using TMVA::Experimental::SOFIE::UTILITY::FillOutput;

FillOutput(tensor_result3, output_tensor_result3, 16);
}

std::vector<float> infer(float const* tensor_input1){
    std::vector<float > output_tensor_result3;
    doInfer(tensor_input1, output_tensor_result3 );
    return {output_tensor_result3};
}
}; // end of Session
} //TMVA_SOFIE_PyTorchModel
#endif // ROOT_TMVA_SOFIE_PYTORCHMODEL

```

3. (rootenv)

utsav@AjatShatru:~/root/tutorials/machine_learning\$ CUDA_VISIBLE_DEVICES="" python TMVA_SOFIE_Keras.py

Sample output:-

```
using TMVA::Experimental::SOFIE::UTILITY::FillOutput;
    FillOutput(tensor_keras_tensor_18, output_tensor_keras_tensor_18, 4);
}

std::vector<float> infer(float const* tensor_input_layer){
    std::vector<float > output_tensor_keras_tensor_18;
    doInfer(tensor_input_layer, output_tensor_keras_tensor_18 );
    return {output_tensor_keras_tensor_18};
}
};    // end of Session
} //TMVA_SOFIE_KerasModel

#endif    // ROOT_TMVA_SOFIE_KERASMODEL
*****

Inference output: { 0.497566f, 0.502434f, 0.502281f, 0.497719f }
```

Exercise 3:-

Setup:-

```
git clone https://github.com/ajatshatru01/SOFIE.git
source ~/root/build/bin/thisroot.sh
cd SOFIE
git checkout gpu/alpaka
mkdir build
cd build
cmake -G Ninja .. \
  -DCMAKE_BUILD_TYPE=Release
ninja
cd ~/SOFIE/src/SOFIE_core/inc/SOFIE
nano ROperator.hxx
nano ROperator_Cast.hxx
nano ~/SOFIE/src/SOFIE_core/src/RModel.cxx
cd ~/SOFIE/src/SOFIE_parsers
nano inc/SOFIE/RModelParser_ONNX.hxx
```

Improvements I have noticed:-

- **ROperator.hxx**

1. Improve const-correctness of operator interface methods
2. Avoid passing strings by value in Generate functions
3. Replace std::set with faster lookup container for fusable operators
4. Decouple GPU backend (Alpaka) from base operator abstraction
5. Replace string-based kernel generation with template kernels

6. Complete OperatorKind toString mapping
7. Improve memory safety by removing unnecessary mutable fields

- **ROperator_Cast.hxx**

1. Improve const-correctness and avoid unnecessary copies in operator interfaces.
2. Add override specifiers to derived operator methods to improve type safety.
3. Replace string-based GPU kernel generation with template-based kernels.
4. Decouple Alpaka backend logic from operator abstraction.
5. Improve kernel launch configuration using device capability queries.
6. Introduce reusable element-wise kernels for operators like Cast, Relu, Sigmoid.
7. Improve operator enum extensibility using dynamic operator registration.
8. Remove redundant checks and improve error messages.

- **RModel.cxx**

1. Fix undefined behavior in GetTensorShape returning reference to temporary objects.
2. Improve const-correctness and avoid unnecessary string copies.
3. Simplify tensor lookup with a unified tensor registry.
4. Replace manual memory pool logic with an arena allocator.

5. Improve operator execution model using an explicit computation graph.
6. Replace string-based code generation with templated kernels.
7. Introduce backend abstraction for CPU/GPU execution.
8. Enable operator fusion using existing tensor lifetime information.
9. Improve exception handling with dedicated error types.
10. Replace `std::cout` debugging with a structured logging system.

- **RModelParser_ONNX.hxx**

1. Improve const-correctness and avoid unnecessary string copies.
2. Replace `vector<bool>` with a safer container for fused operator tracking.
3. Merge `OperatorKind` enum with dynamic operator registry.
4. Move tensor metadata management into `RModel` instead of parser maps.
5. Improve error handling for unsupported ONNX operators.
6. Add operator metadata to improve validation and extensibility.
7. Decouple backend execution (CPU/GPU) from parsing logic.
8. Introduce operator registration helpers to reduce boilerplate.

Exercise 4 and 5:-

For exercise 4, I have added Alpaka GPU implementation in ROperator_Elu.hxx

For exercise 5, I have added GPU methods for ROperator_BatchNormalization.hxx.

The entire code, along with test have been pushed to my github repo:-

<https://github.com/ajatshatru01/SOFIE/tree/gpu/alpaka>

Mentors can see the commits here:-

<https://github.com/ML4EP/SOFIE/compare/gpu/alpaka...ajatshatru01:SOFIE:gpu/alpaka>

Setup steps to run tests:-

cd ~/SOFIE/build

ctest --output-on-failure

Sample run:-

```
utsav@AjatShatru:~/SOFIE$ cd ~/SOFIE/build
utsav@AjatShatru:~/SOFIE/build$ ctest --output-on-failure
Test project /home/utsav/SOFIE/build
  Start 1: src-SOFIE_core-test-TestBatchNormalizationOperator-build
1/4 Test #1: src-SOFIE_core-test-TestBatchNormalizationOperator-build ... Passed    0.06 sec
  Start 2: src-SOFIE_core-test-TestBatchNormalizationOperator
2/4 Test #2: src-SOFIE_core-test-TestBatchNormalizationOperator ..... Passed    0.03 sec
  Start 3: src-SOFIE_core-test-TestEluOperatorSimple-build
3/4 Test #3: src-SOFIE_core-test-TestEluOperatorSimple-build ..... Passed    0.03 sec
  Start 4: src-SOFIE_core-test-TestEluOperatorSimple
4/4 Test #4: src-SOFIE_core-test-TestEluOperatorSimple ..... Passed    0.02 sec

100% tests passed, 0 tests failed out of 4

Total Test time (real) =  0.16 sec
```

